第一章 第二讲 · 实践部分：登录模块设计

> 数据结构教学案例整理 | 案例：学生管理系统登录、教学质量评价系统登录

案例一：学生管理系统登录模块设计

1. 案例描述

设计一个学生管理系统登录模块。要求：

- 初始化 N 个用户信息（用户信息包含：账号 id、密码 pwd）。
- 输入当前用户的账号和密码进行验证。
- 验证成功输出“登录成功！”；验证失败输出“账号或者密码错误！”。

2. 案例分析（数据结构设计）

- **数据元素：** 一个用户的信息。
- **数据项：** 账号（长整型 `long id`）、密码（字符数组 `char pwd[6]`）。
- **数据结构：** 采用结构体数组实现，定义结构体类型 `user`，并使用数组 `user users[50]` 存储多个用户信息，使用指针 `user *p` 提高访问效率。

3. 核心C语言代码清单

```c
#include <stdio.h>
#include <string.h>

// 1. 定义数据结构
typedef struct {
    long id;
    char pwd[6];
} user;

user users[50]; // 数组定义，用来存储多个用户信息

// 2. 初始化函数
void init(int n) { // 输入用户信息
    user *p;
    int i;
    p = users; // 指针的应用
    printf("请管理员输入用户信息:\n");
    for(i = 0; i < n; i++) {
        scanf("%ld,%s", &p[i].id, p[i].pwd);
    }
}

// 3. 主函数与验证逻辑
int main() {
    long stuid;
```

```
    char stupwd[6];
    int i, n;

    printf("请输入用户数量: ");
    scanf("%d", &n);

    init(n); // 调用输入用户信息函数

    printf("请您输入账号和密码:");
    scanf("%ld,%s", &stuid, stupwd);

    for(i = 0; i < n; i++) {
        // 验证账号与密码
        if(stuid == users[i].id && strcmp(stupwd, users[i].pwd) == 0) {
            printf("登录成功! \n");
            break;
        }
    }

    if(i == n) {
        printf("账号或者密码错误! \n");
    }
    return 0;
}
```

案例二：教学质量评价系统登录模块设计（技能提高）

1. 案例描述

设计一个教学质量评价系统登录模块。要求：

- 初始化 N 名教师信息（教师信息包含：账号 id、密码 pwd、权限 pms）。
- 输入当前教师的账号、密码和权限进行验证。
- 验证成功，继续判断权限：若权限为 1，输出“教学督导”；若权限不为 1（如为 2），输出“普通教师”。
- 若验证失败，输出“账号或者密码错误，登录失败！”。

2. 案例分析（数据结构设计）

- **数据元素：** 一位教师的信息。
- **数据项：** 账号（长整型 `long id`）、密码（字符数组 `char pwd[6]`）、权限（整型 `int pms`）。
- **逻辑结构：** 在案例一的基础上增加了选择嵌套结构（`if...else if...else`），在验证通过后再根据 `pms` 的值进行二次权限判断。

3. 核心C语言代码清单（部分）

```
#include <stdio.h>
#include <string.h>

// 1. 定义数据结构
```

```c
typedef struct {
    long id;
    char pwd[6];
    int pms; // 新增权限数据项
} user;

user users[50]; // 存储多名教师信息

// 2. 初始化函数
void init(int n) { // 输入教师信息
    user *p;
    int i;
    p = users; // 指针应用
    printf("请管理员输入教师信息:\n");
    for(i = 0; i < n; i++) {
        scanf("%ld,%s,%d", &p[i].id, p[i].pwd, &p[i].pms);
    }
}

// 3. 主函数与权限验证逻辑
int main() {
    long tid;
    char tpwd[6];
    int tpms;
    int i, n;

    printf("请输入教师数量: ");
    scanf("%d", &n);

    init(n); // 调用输入教师信息函数

    printf("请您输入账号、密码和权限:");
    scanf("%ld,%s,%d", &tid, tpwd, &tpms);

    for(i = 0; i < n; i++) {
        // 账号与密码校验
        if(tid == users[i].id && strcmp(tpwd, users[i].pwd) == 0) {
            // 权限二次判断（嵌套选择结构）
            if(tpms == 1) {
                printf("教学督导\n");
            } else {
                printf("普通教师\n");
            }
            break; // 找到匹配项后终止循环
        }
    }

    // 循环结束仍未找到匹配
    if(i == n) {
        printf("账号或者密码错误，登录失败！\n");
    }
    return 0;
}
```

教学知识点总结

这两个案例很好地串联了 C 语言基础及数据结构入门的核心知识：

- **数据封装：** 使用 `typedef struct` 将不同类型的数据（账号、密码、权限）组合成一个逻辑单元。
- **线性存储：** 使用结构体数组 `users[50]` 实现数据的顺序存储。
- **指针操作：** 使用指针 `user *p` 指向数组首地址，用 `p[i]` 或 `(p+i)` 进行高效遍历。
- **字符串处理：** 密码作为字符串不能直接用 `==` 比较，必须使用 `string.h` 库中的 `strcmp()` 函数。
- **算法逻辑：** 结合 `for` 循环与 `if` 判断实现“线性查找（Linear Search）”，在查找成功时跳出循环，并使用 `i == n` 作为未找到匹配项的边界判定条件。